

Decentralized, Browser-Based Docker IDEs for C++ and x86-64 Assembly Education: An Evaluation on the Laptops Students Actually Own

Sean Cuenco · Faculty Mentor: James Andro-Vasko, Ph.D. · Department of Computer Science, University of Nevada, Las Vegas

Summer Undergraduate Research Fellowship (SURF) 2026 · Office of Undergraduate Research, UNLV

Code, handouts, test script, raw results:
github.com/seancnc003/UNLV Docker IDEs

Images:
hub.docker.com/r/seancnc/unlv-cpp-ide
hub.docker.com/r/seancnc/unlv-x86-ide

Abstract

Students in UNLV’s introductory C++ (CS 135) and x86-64 assembly (CS 218) courses lose the first week of the semester installing a compiler, assembler, debugger, and editor on dozens of different laptops, and owners of Apple Silicon Macs often cannot run the course virtual machine at all. This project packages each course’s exact toolchain and a browser-served Visual Studio Code editor into a Docker image that runs entirely on the student’s own machine: one 552 MB download, one command, no server, no account, no telemetry. The fellowship asked whether that promise holds on the computers students actually own, and what the design gives up compared with shared SSH servers, course virtual machines, cloud editors such as CS50 on GitHub Codespaces, and monitored platforms such as CodeDive. One unmodified test script performed a student’s first day and then built and ran four real CS 218 assignments on four hosts: Intel Linux and ARM Linux rented from AWS, Windows 11 rented from Azure, and an M1 MacBook Pro. The same image ran every assignment correctly on all four, started in 0.5 s natively and 5–7 s under emulation, and idled at 55–260 MiB. The one limit is precise: the gdb debugger works on Intel/AMD hosts and is silent under both ARM translation layers, a trade made deliberately to keep the course’s real x86-64 instruction set. A classroom study is the next step.

1 Background: week one is lost to setup

Every systems course starts the same way: a compiler, an assembler, a debugger, and an editor must work on dozens of different laptops before anyone learns anything. Prior studies measured 75–100 minutes of class time per semester lost to installs [6], support traffic dominated by environment mismatch rather than course content [3], and students with M1/M2 MacBooks unable to run the x86 virtual machines their courses assumed [4]. Several hundred CS 135 and CS 218 students a year are affected, and the friction falls hardest on the cheapest and newest machines.

Research questions answered before deployment

RQ1 Reproducibility. Do the images run the documented workflows and real CS 218 assignments?

RQ2 Portability. Which operating systems and processors can run the assembly environment?

RQ3 Resource cost. Startup, storage, memory, compile time, and the emulation overhead on ARM.

RQ4 Persistence. Do student files survive replacing the container?

RQ5 Architecture fidelity. How far does an amd64-only image, including its debugger, work on ARM?

UNLV’s own path, one step behind the literature

1. Shared login servers over SSH (PuTTY)

bobby, sally, cardiac: two decades of departmental Linux hosts and a submit script that required logging in even from the lab. Now decommissioned; the wiki documenting them was deleted in a 2025 site rebuild [12].

2. Virtual machines, one per course

CS 218 today: a 19 GB VirtualBox image for Intel and an experimental Fall-2023 image for Apple Silicon that emulates a whole x86-64 computer, so slowly the course warns VS Code “may not be usable.” Official advice for M1/M2 owners: go to the TBE-A311 lab with a USB drive.

3. A container on the student’s own laptop (this project)

One 552 MB download, one command, VS Code in a browser tab. Files stay in a normal folder on the laptop. No server, no account, no VPN; works offline.

2 Methods: what we built

Two Docker images, one per course, each packing code-server (VS Code served to the browser) with the exact toolchain the course expects: g++ for CS 135; yasm, nasm, ld, and gdb for CS 218. The instructor pins every version. The x86 image is deliberately amd64-only so students assemble and run real x86-64 binaries; on ARM hosts Docker runs it through a translation layer. Every AI feature in the editor is disabled per course policy.

```
docker run -p 127.0.0.1:8218:8080 -v ~/UNLV/x86-workspace:/home/coder/workspace seancnc/unlv-x86-ide
```

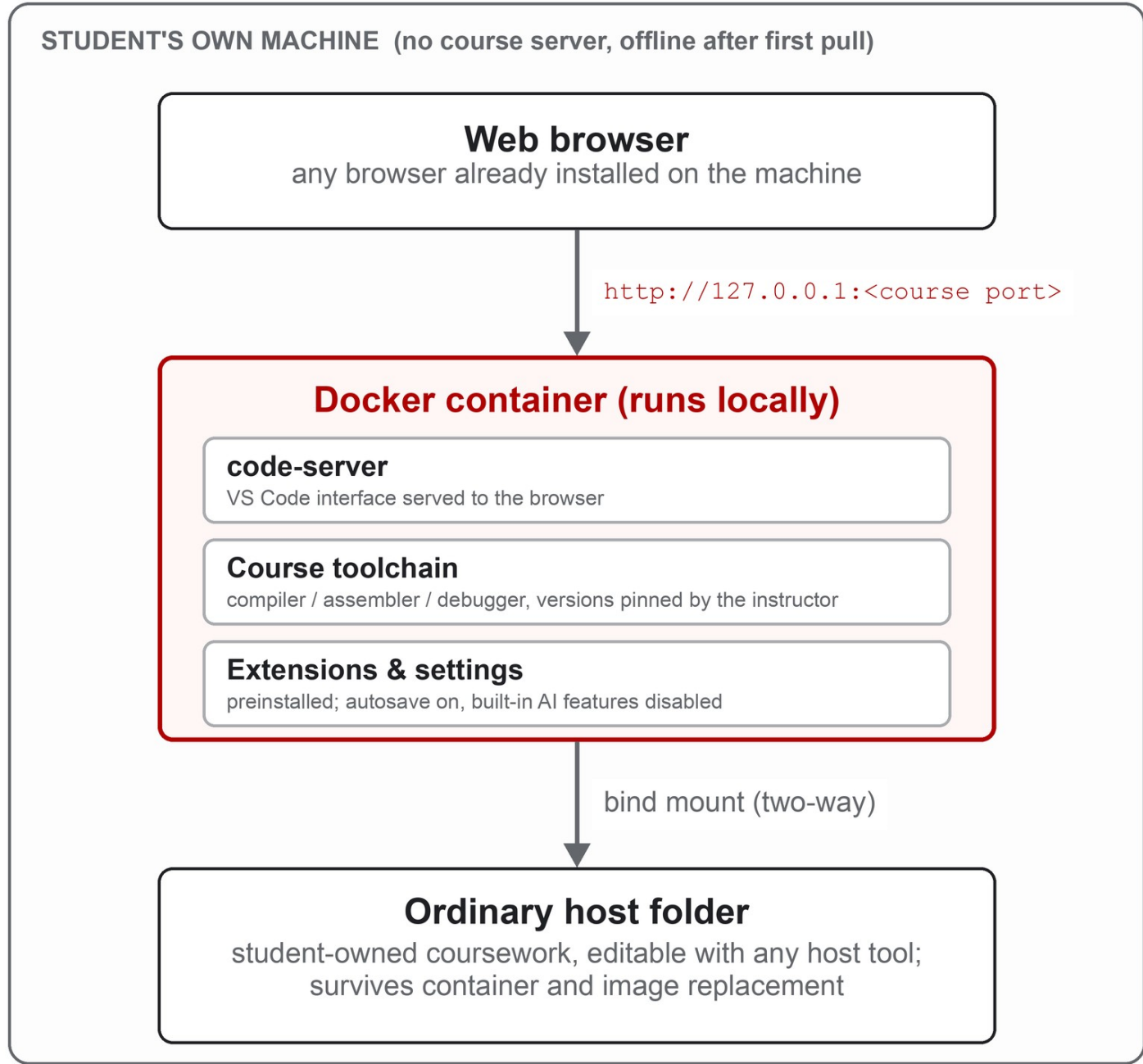


Figure 1. One course IDE. Everything runs on the student’s machine; coursework lives in an ordinary host folder that survives deleting or updating the container. The only network dependency is the first download.

2.2 Evaluation: same exam, four kinds of computer

We cannot borrow one of every laptop students own, so we rented them (Table 1). On each host the same unmodified script (ci-test.sh) performed a student’s first day and more: a timed download that records the image digest; a timed start until the editor answers; a starter-file check; three assemble-link-run cycles; a scripted gdb probe; idle memory; a timed build and run of four real CS 218 assignments (ast3, ast04, ast06, ast12, the last multithreaded) with peak memory sampled; and destruction and recreation of the container to check that files survive. Every number on this poster is transcribed from the JSON the script writes.

Table 1. The four hosts. Cells 1 and 2 are same-size Intel/ARM siblings, so their comparison isolates emulation.

Cell	Host	OS / Docker	x86 image
1 Linux amd64	AWS m8i.large, Intel Xeon, 2 vCPU / 8 GB	Ubuntu 24.04, Docker Engine 29.1	native
2 Linux arm64	AWS m8g.large, Graviton (ARM), 2 vCPU / 8 GB	Ubuntu 24.04, Docker Engine 29.1, QEMU	emulated
3 Windows amd64	Azure D4s_v5, Intel Xeon, 4 vCPU / 16 GB	Windows 11 Pro 24H2, Docker Desktop, WSL2	native
4 macOS arm64	MacBook Pro 14-inch (2021), M1 Pro, 16 GB, owned	macOS 26.4, Docker Desktop 29.2 (Rosetta)	emulated

3 Results

4 of 4

hosts ran every real CS 218 assignment correctly, including the multithreaded one

0.5 s

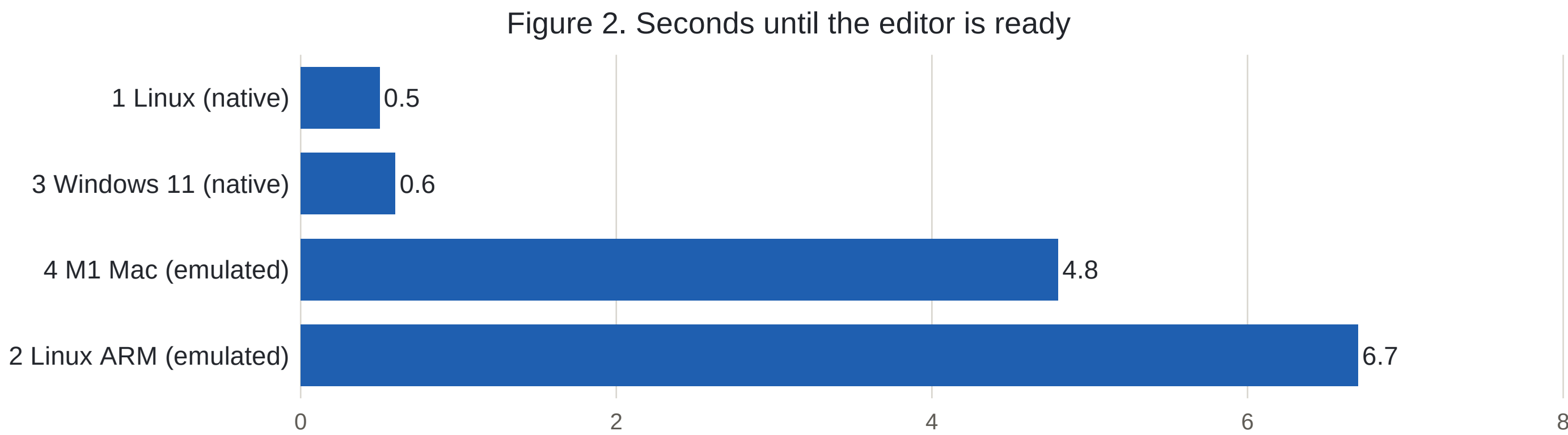
until the editor is ready on Intel/AMD; 5–7 s under emulation; a course VM boots in minutes

552 MB

one-time download, byte-identical on every host, versus a 19 GB virtual machine

Table 2. Selected results for the x86 image (all values from the unmodified script).

Metric	1 Linux native	2 Linux ARM emulated	3 Windows 11 native	4 M1 Mac emulated
Debugger (gdb) probe	working	broken	working	broken
Start until editor ready (s)	0.5	6.7	0.6	4.8
Idle memory (MiB)	54	255	56	263
Peak memory, coursework (MiB)	n/a	334	90	358
ast06 build (s)	0.2	2.8	0.5	2.5
ast12 build (s)	0.3	3.6	0.6	3.1
All four assignments pass	yes	yes	yes	yes
Files survive replacement	yes	yes	yes	yes
Checks passed / failed	13 / 0	12 / 0	13 / 0	12 / 0



The support boundary is a translator boundary, not an operating-system boundary. Windows and Linux on Intel/AMD are native and get everything; Windows 11, which dominates student laptops, was the strongest host in the matrix. Apple Silicon works because Apple supplies its Rosetta translator to Docker’s Linux VM. Emulation costs about one order of magnitude on builds (10–14×), yet every emulated build finished in under four seconds. The one loss is gdb: under translation a debugger script still “succeeds” and writes its file, but with no register values, so the handout tells Apple Silicon owners to produce debugger deliverables on an Intel machine. On ARM Linux the stock Ubuntu 24.04 QEMU crashed translating the editor itself; Docker’s own translator passed every check, so the fix is one line in the handout. **Where this sits.** CS50 took the container and browser editor to GitHub’s servers [1]; this project puts the same pair back on the student’s machine, and the measurements show the trade is affordable. Cadenas et al. kept a working debugger by changing the course’s instruction set to whatever the emulator provided [5]; this design keeps the real x86-64 instruction set and loses the debugger on ARM. Native hardware gets both; nothing on ARM yet does, and this evaluation makes that trade-off measured and explicit.

4 Discussion: every approach hands the setup burden to someone

Approach	Who carries the burden	What it costs the student
Shared login servers over SSH (UNLV bobby / sally / cardiac)	The department	Network and VPN dependence; the environment and its documentation vanish when the servers are retired
Course virtual machines (CS 218 images; CS50 Appliance [2]; Reading’s QEMU lab [5])	The student’s hardware	Multi-gigabyte images, fixed RAM reservations, slow boots, disk corruption; unusable or very slow on Apple Silicon
Cloud-hosted browser IDE (CS50 on GitHub Codespaces, 700,000+ users [1])	A vendor, a grant, an account	Internet required; the environment disappears with the account; students leave “not sure how to set up an IDE on my computer”
Monitored classroom platform (CodeDive, South Korea [8])	The institution’s cluster, and the student’s privacy	Every process and every one-second typing pause recorded (24,845 snapshots from 95 students in two weeks); consent that cannot realistically be withheld
Local container with browser editor (this project; cf. UCSD [3], Florida Tech [4])	The student, once: install Docker Desktop	One download and one command; offline afterward; files stay on the laptop; debugger unavailable on Apple Silicon and ARM

5 What comes next

- A classroom study in CS 218 and CS 135 with IRB approval: how many students reach a working setup without help, how many staff minutes it saves, and how students feel about it, using instruments adapted from [3, 4, 6]. A three-student high-school pilot has already run on the C++ image.
- A hybrid assembly image for ARM that runs the editor natively and translates only the student’s program, which could restore the debugger on Apple Silicon and Windows-on-ARM.
- Reuse the one-script, rent-four-computers method to vet environments for other courses, starting with Operating Systems.

Supported by the UNLV Office of Undergraduate Research Summer Undergraduate Research Fellowship. Every number on this poster comes from the published test script’s output files, archived in the repository under tests/record-results.

References (numbered as in the written report)

- [1] D. J. Malan. Containerizing CS50: Standardizing Students’ Programming Environments. ITCSE 2024.
- [2] D. J. Malan. From Cluster to Cloud to Appliance. ITCSE 2013.
- [3] S. Valstar, W. G. Griswold, L. Porter. Using DevContainers to Standardize Student Development Environments. ITCSE 2020.
- [4] K. Fernslied, T. O’Connor, S. Sudhakaran, N. Nur. Lightweight Symphony: Reducing CS Student Anxiety with Standardized Docker Environments. SIGITE 2023.
- [5] J. O. Cadenas et al. Virtualization for Cost-Effective Teaching of Assembly Language Programming. IEEE Trans. Educ. 58(4), 2015.
- [6] D. P. Harvie, J. R. Cody, C. Morrell, T. T. Estes. Using Virtual Machines to Enhance the Educational Experience. SIGITE 2019.
- [8] H. Park et al. CodeDive: A Web-Based IDE with Real-Time Code Activity Monitoring. Applied Sciences 15(19), 2025.
- [12] UNLV Dept. of Computer Science. Student Center: Remote Access and File Storage. tux.cs.unlv.edu, accessed Aug. 2026.